源码驱动的架构实战

Octop 架构解剖

从需求到运行时

参考本地 SAM 系列的讲解方法
前言 · 17 章 · 附录

Octop 1.0.3b1
890111b7879665840414ca28ab168b05b0db0921
采集日期 2026-10-27

独立分析，非官方文档。静态源码研究与离线教学验证，不代表生产压测或安全审计。

全书目录

1. 前言：沿一次任务读懂平台
2. 需求分析：先确定系统要保住什么
3. 总体架构：单进程，不是单模块
4. 启动与退出：谁把系统装起来
5. Agent 装配：配置如何变成能力
6. 身份与会话：四种 ID，四条边界
7. 存储与历史：不要寻找唯一数据库
8. 工作区后端：远程文件不等于远程执行
9. 一次对话：从 WebSocket 到模型再回来
10. 记忆与知识库：两条不同的检索路径
11. 工具与扩展：Skill、Plugin、Connector 和 MCP
12. 多 Agent：房间共享，状态分离
13. 安全边界：能聊天不等于能管理
14. 前端架构：流式对话是一台状态机
15. 跨端协作：ACP 与 Bridge 各管什么
16. 部署与验证：把恢复当成一项功能
17. 架构评价：何时保留单体，何时拆分
18. 代码实战：用最小宿主验证四条边界
19. 附录：源码索引、术语与核验路线

00

前言：沿一次任务读懂平台

从一个能核对的任务开始

一家小型出版工作室希望把作者访谈整理成选题材料。编辑林岚在浏览器里提出：“查阅访谈附件和选题规范，请研究专家整理背景，再给我一份可以审阅的提纲。涉及外发时先问我。”

这个案例是本书虚构的教学场景，不是 Octop 自带的业务流程。它足够小，却跨过平台里最重要的边界：用户身份、Agent 工作区、附件、知识检索、模型工具调用、专家协作、人工审批和消息归档。我们用同一场景问系统：谁执行、读哪份状态、写到哪里、失败后还剩什么。

如果只看功能列表，这些能力像互不相干的插件。沿一次任务追下去，会看到 Octop 的主要工作是把可复用运行库接成一个能自托管的产品。

分析版本与证据等级

本书依据 TencentCloud/Octop 提交 `890111b7879665840414ca28ab168b05b0db0921`，其 `pyproject.toml` 标记版本为 `1.0.3b1`。采集日期是 2026-10-27。网页中的源码链接固定到该提交，不随默认分支漂移。

全书区分三类表达：

表达	含义	怎样使用
源码事实	本次快照可定位的类、调用和检查	打开本章源码入口复核
架构分析	由组件关系推导的收益、约束和风险	当作有依据的解释，不当作实测
演进建议	面向更大负载的改造方向	先测量，不假设项目已实现

这次工作是静态架构研究，没有启动完整 Octop，没有调用真实模型、IM、Bridge 或 PostgreSQL 服务。书中提到测试文件，是说明仓库提供了相应的验证入口，不代表本次已运行并全部通过。

Octop 依赖独立发布的 octop-harness、octop-memory、octop-gateway 和 octop-browser。本书重点研究宿主如何使用它们，不把宿主的一段配置当成已审计外部库全部内部行为的证据。

先纠正三种阅读习惯

第一，不以 README 的一棵树替代当前对象图。中文 README 仍有“按用户的 HarnessAgentManager”和“按 Agent 的 CronManager”的旧表达；架构文档与 server.py 显示的是进程级 AgentManager、Gateway 和 CronManager。正文采用当前源码结构。

第二，不从文件名推断协议。前端仍有叫 sseHelpers 的文件，普通聊天却已走双向 WebSocket；部分审批恢复仍用 SSE。迁移期间的旧命名不是传输协议的事实来源。

第三，不把“可选 PostgreSQL”理解成“平台完全无本地状态”。知识库索引仍有本地 SQLite sidecar；分段历史归档也有 SQLite-only 限制。选型必须按数据类别检查。

两条阅读路线

第一次接触 Agent 平台，可按顺序读。先读需求与技术架构，再看身份、数据模型与工作区，随后沿一次请求进入工具与协作，最后讨论安全、界面与运维。已经维护相似项目，可以先读第八章的一次请求，再跳到第六章存储与第十一章团队，最后回读装配代码。

每章的练习都可以先在纸上推演。附带的标准库示例只演示身份、会话、审批和状态之间的关系，不模拟模型智能，也不是 Octop API 的替代实现。

▼ 阅读约定：图、示例与源码边界

文中的箭头表示控制流或数据流，图注会说明，不表示每一步属于同一事务。包含类名的片段优先作为导航地图。标记为“教学模型”的伪代码不承诺可以直接粘贴到 Octop 中运行。案例里的姓名、材料和工作室均为虚构。

源码入口

- `pyproject.toml`
- `README_CN.md`

docs/architecture.md

动手检查

查找版本号、提交哈希和正文范围。假如半年后 main 分支把聊天协议改了，你应该改动本书中的哪一项信息，才能不让旧书变成错误的“当前文档”？

答案提示： 保留旧提交的书仍然成立；更新时必须重新固定快照并复核调用链，而不是只改封面日期。固定版本描述的是“这个实现”，不是“永远如此”。

01

需求分析：先确定系统要保住什么

为什么不从模型 SDK 开始

工作室最初只需要“上传一份访谈，生成一份提纲”。用几十行脚本调用模型确实能完成一次生成。但下一周就会出现新问题：另一位编辑能不能用同一个研究专家？她会不会看到林岚的聊天？浏览器关了，专家还该不该继续工作？外发稿件前在哪里确认？

这些问题不是选哪家模型能解决的。它们决定平台需要哪些持久对象、入口适配和授权规则。本章把项目已有能力整理为一份教学 PRD，不声称这是 Octop 团队最初的内部需求文档。

用户与业务边界

角色	需要完成的事情	不应顺带获得的权力
普通编辑	聊天、上传资料、管理自己的专家	读取其他人的私人线程
专家所有者	配置模型、技能、工作区和分享范围	因分享而暴露所有历史
管理员	初始化实例、管理用户和全局配置	无须审计的任意外部副作用

角色	需要完成的事情	不应顺带获得的权力
外部协作者	经授权渠道或远程桥使用专家	绕过专家所在侧的身份检查

管理员在代码里有多处 bypass，这是现有权限模型的事实，不表示所有动作都天然适合授予管理员，更不表示面向陌生租户时已具备强隔离。

核心产品边界是家庭、小团队和个人的自托管助手。不要把“支持多用户”直接升级成“已经满足公共云多租户 SaaS 的隔离、配额和可用性要求”。

把需求变成架构决策

业务需求	引出的设计	当前证据
安装负担低	单 Python 核心服务，打包 Dashboard	ADR 001、pyproject.toml
多入口调用相同专家	统一 InboundMessage 与处理入口	chat/turn.py、GlobalProcessor
专家可共享，私人聊天不串	专家权限与线程归属分别校验	common/agent.py、resolve_thread_id
关页面不等于停止任务	turn 归宿主，连接只订阅	chat/ws.py 断线清理
能用本地文件，也能接远程存储	BackendWorkspace 抽象	backend/ 与工作区 I/O 文档
需要恢复、展示和长期记忆	checkpoint、历史、Memory 分工	ADR 002、history/、memory/
多位专家合作	主持人编排，成员独立状态	expert-teams.md

业务需求	引出的设计	当前证据
危险操作先审批	工具策略、HITL 与恢复通路	security/、gateway/hitl/

这张表也用来判断新增功能放在哪里。若只是新增一个 IM 平台，应优先适配通道协议；若改变“谁能读哪个线程”，则必须进入业务授权边界，不能只改前端菜单。

一次任务的验收标准

给工作室设计验收时，不先写一个未经测试的 QPS。先确定行为正确：

1. 林岚能创建或选择自己的研究专家，并上传访谈。
2. 请求中的附件引用最终指向该专家可读的工作区文件。
3. 研究结果能持续展示，切换页面不强行取消任务。
4. 另一用户可以使用已共享专家，但不能指定林岚的 thread_id 读取她的私人对话。
5. 外发工具触发审批后，不因用户点两次就执行两次。
6. 模型请求失败时，界面和历史不得显示“任务成功完成”。
7. 重启之后可以读取已提交历史；正在执行的工作按明确语义中断，而非无依据地冒充恢复完成。

第 5 项是验收目标，不能只因项目存在 HITL 就认定所有外部工具已幂等。第 7 项也必须区分历史可读、checkpoint 可恢复与业务动作可重试。

非功能需求要带测量方法

“快”至少拆成首帧等待、首 token 等待、工具耗时、结束后归档耗时。“稳定”要拆成断线重连、模型超时、磁盘写满、进程退出、索引失败和备份恢复。不同测试验证的是不同保证。

一个可复用的验收记录应包含输入用户、agent_id、session_key、thread_id、预期动作、实际落盘与外部副作用。这样故障能回到具体边界，而不是只留下“回答不对”的截图。

▼ 参考 SAM 系列的范围

本章参考本地 SAM 支付系列先梳理场景、业务规则和验收标准，再推导技术方案的组织方式。没有把支付系统的微服务数量、交易指标或事故经历迁移到 Octop。Octop 的需求描述由公开功能与源码反向整理，属于教学分析。

源码入口

- `README_CN.md`
- `docs/architecture.md`
- `src/octop/api/common/agent.py`
- `src/octop/api/routers/chat/ws.py`

动手检查

产品提出：“共享一个专家后，所有编辑默认看到这个专家的所有聊天，方便合作。”这是一处界面需求，还是权限模型变更？

答案提示： 是权限和产品语义变更。现有模型区分可用专家与线程归属；若要共享历史，必须明确授权对象、范围、撤销与审计，不能仅让前端列表多展示几行。

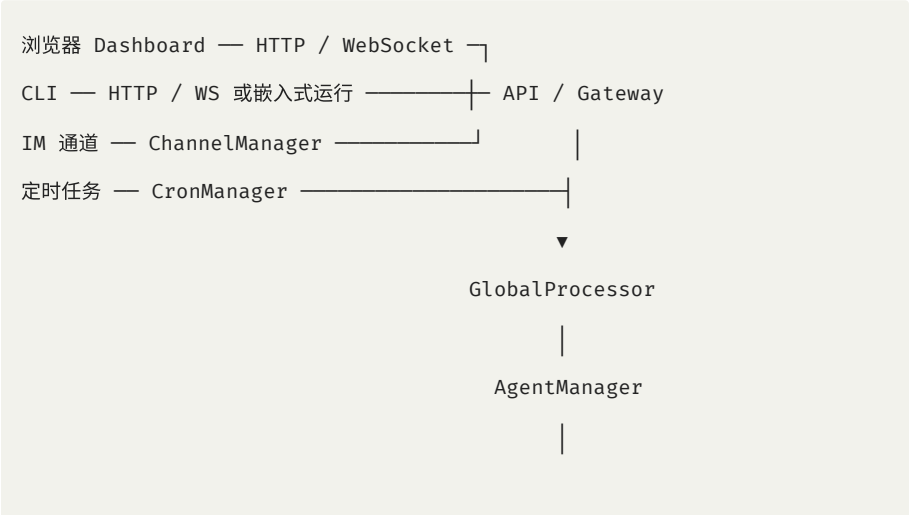
02

总体架构：单进程，不是单模块

Octop 首先是一个宿主

林岚看到的是聊天界面，维护者看到的却是一组持续运行的组件：认证、Agent 注册表、IM 连接、定时任务、消息订阅和数据库。Octop 把它们装进同一个 Python 服务，而不是要求用户先部署消息队列、worker 与独立控制台后端。

把它归类为“模块化单体”比“聊天机器人”更准确。这是架构分析：它具有多种领域能力和明确的库边界，但核心运行时由一个进程持有。单进程减少部署部件，不会自动减少模块之间的复杂度。





图表示入口汇聚关系，不表示每种 CLI 命令都会经过 HTTP，也不表示每条路径都按图中全部节点逐一调用。

四个产品边界

对外表面包含 Dashboard、Click CLI 和 HTTP/WS API。API 层处理认证、参数、状态码和传输，不应该自行拼装一套与 IM 不同的模型循环。

宿主业务层位于 infra。这里的 AgentManager 是进程级注册与装配入口；Gateway 连接渠道、线程与消息；CronManager 管理调度；SharedServices 提供仓储与公共服务。文件夹名叫 infra，但其职责不只是数据库驱动，还包括相当多产品逻辑，不能机械套进“纯基础设施层”。

可复用库承担不同任务。octop-harness 提供 Agent 运行时与工具编排；octop-gateway 提供通道协议；octop-memory 提供记忆能力；octop-browser 提供浏览器相关能力。依赖名可以从 pyproject.toml 核实。详细内部算法需要另读相应库，不能从宿主的 import 推演出全部实现。

存储层也不是一个部件。控制面记录用户和配置；运行时 checkpoint 保存图状态；工作区承载内容文件；历史与知识索引还有自己的保存形式。这些边界到存储与历史一章再展开。

为什么不先拆服务

ADR 001 给出的背景是自托管用户：安装一个包，初始化，再启动服务。把 Redis、Celery 或单独 worker 设为必需，会把一个人的安装问题变成多服务运维问题。

Agent 请求经常等待模型、网络或工具返回，异步 I/O 可以在等待期间处理其他请求。这支持单进程选择，但不说明 CPU 密集型 PDF/OCR、向量扫描或同步数据库调用没有成本。事件循环里出现阻塞，仍会影响整个实例。

另一个好处是对象间可以直接传引用。CronManager 可以得到 Gateway，AgentManager 可以得到仓储与策略，无须再设计远程接口。代价也来自同一点：这些对象、锁和订阅只存在于本进程，多开实例不会自动共享。

PostgreSQL 没有改变进程模型

ADR 002 引入可选 PostgreSQL 控制面。它改变的是持久化位置和驱动，不是给所有内存组件提供分布式一致性。文档明确不承诺多实例同时写。

例如运行两个进程读取同一个 cron 表，每个进程都可能恢复自己的调度器。数据库里只有一条任务，不代表实际只有一个调度所有者。要做横向扩展，需要任务认领、租约、幂等和消息路由，而不只是换连接串。

▼ 源码定位与解释边界

从 infra/server.py 的 _boot_runtime 开始画对象图，再看 launch.py 如何挂 FastAPI。docs/architecture.md 的全局注册表说明与代码一致。README_CN 的旧树仅作为文档漂移证据，不用来断言当前运行结构。

源码入口

- `docs/architecture.md`
- `docs/adr/001-single-process-model.md`
- `docs/adr/002-database-backends.md`
- `src/octop/infra/server.py`

动手检查

工作室有 12 位编辑，打算把数据库换成 PostgreSQL，再启动三个相同服务以提高并发。指出两种不能仅靠数据库解决的状态。

答案提示： WebSocket 订阅与正在运行的 turn、Cron 调度器、团队在途派工和取消信号都带进程内状态。要先设计所有者与路由；“12 人”本身也不足以证明需要拆分，应测量同时活跃任务及尾延迟。

03

启动与退出：谁把系统装起来

两个装配入口，各有职责

启动命令最终进入 launch.py 的 run_foreground。这里同时知道 OctopServer 与 FastAPI，并把它们接给 uvicorn。业务组件的装配放在 infra/server.py，HTTP 应用组装放在 api/app.py。

这样的边界防止业务核心为了启动渠道而反向依赖 HTTP 路由，也让嵌入式 CLI 可以使用 OctopServer 而不必总启动 Web 服务。所谓 composition root，就是集中决定“创建谁、把谁传给谁”的位置，不是一个新的业务服务。

从文件到可接收请求的系统

常规路径可以压缩成下面的依赖顺序：

```
PathLayout 与配置
  → 专家 / 子 Agent 目录，插件播种与加载
  → 打开控制面数据库，执行迁移
  → SharedServices 与 JWT 所需材料
  → AgentManager、历史/轨迹服务、Gateway、Cron 等
  → AppRuntime
  → FastAPI 应用与监听器
```

这是一张依赖地图，不是源码所有语句的逐行列表。启动还包含日志、资源限制、浏览器配置、知识库任务恢复、Bridge 自动重连等支线。

源码提供一个容易漏掉的分支：首次安装时，数据库后端尚未由设置向导选定，should_defer_control_plane_db 可以让服务暂缓打开控制面。此时“进程活着”不等于“所有业务依赖已就绪”。bind_control_plane 在用户完成选择后再创建仓储和运行时。

林岚第一次打开页面看见向导，并不意味着服务启动失败。它进入了一个有意设计的未绑定状态。

配置不是读一次就结束

配置来源包括文件与环境变量，Agent 和 Provider 的有效配置又来自控制面。设置向导换库还有空库条件下的重绑定路径。运行时组件因此提供 replace_* 或类似的更新接口，而不是让数据库模块随意修改其他对象的私有字段。

这个安排限定了切换时机，以及切换后必须保持一致的引用，并不支持任意热切换。ADR 002 把已有用户的运行环境与空库 setup 区分开；分段历史归档又会加上额外限制。不能把管理界面上有个数据库选项理解成随时在线迁移所有状态。

启动失败要区分层次

模型未配置、某个 Agent 的远程后端失联、数据库迁移失败，影响范围不同。数据库连接与迁移是许多业务的基础；单个 Agent 的构建则可以有自己的失败状态和超时约束。AgentManager 中可以找到启动超时与有界重载相关代码。

对工作室来说，研究专家暂时不可用不应被误报成全部历史丢失。故障页面和日志至少要分清：宿主未就绪、用户未初始化、专家未运行、当次模型请求失败。

本书没有运行这些失败实验，这里给出的是根据装配依赖推导的排障顺序。

退出不是直接杀进程

server.stop 会依次处理主动关怀调度、cron、gateway、Agent 注册表与用户侧资源，并关闭历史存储与数据库。退出过程中还涉及连接器会话和后端浏览会话的清理。

为什么先停调度再关 Agent？因为调度器继续派活时，运行资源可能已开始销毁。正确的退出过程需要停止接纳、处理或终止在途工作、刷新持久化、释放连接。源码的关闭序列提供了基础，但不要由此推定它对所有外部副作用提供了精确一次保证。

▼ 在纸上画启动状态机

可用未启动、已启动但数据库未绑定、数据库已绑定、业务运行中、关闭中、已关闭六个状态分析。它是教学抽象，不是源码枚举。为每条边标注触发者与失败时仍有效的资源，能很快发现“只判断进程是否存活”的健康检查遗漏。

源码入口

- `src/octop/launch.py`
- `src/octop/infra/server.py`
- `src/octop/infra/db/factory.py`
- `src/octop/infra/db/rebind.py`

动手检查

首次设置选择 PostgreSQL 时连接失败，能否先创建管理员，再把用户表搬过去?

答案提示： 不应跳过目标数据库成功绑定这个前提。否则管理员写到哪一个控制面、后续 JWT 与仓储读哪一个控制面都会变得含糊。应按向导约束解决连接，再完成初始化，而不是临时发明迁移步骤。

04

Agent 装配：配置如何变成能力

Agent 行不是 Agent 对象

控制面中的 Agent 行记录身份、归属、启用状态、模型选择与配置。真正执行模型循环的是通过 octop-harness 创建的运行对象。两者之间由 Octop 的 AgentManager 负责转换。

林岚把研究专家的默认模型改了，首先改变的是持久配置。正在执行的任务是否立即改变、下一轮是否使用新模型、是否需要重建运行时，是另外三个问题。把“配置保存成功”等同于“所有活跃对象已更新”会产生难以解释的故障。

当前 manager.py 很大，既管生命周期，也负责模型、工具、策略与工作区装配。阅读时应沿一个方法的输入输出走，不必从第一行读到最后一行才建立结构。

装配出一份 HarnessAgentConfig

_build_harness_config 附近是理解能力来源的关键。它把若干独立来源合到 HarnessAgentConfig：

来源	装配结果	需要留意
Agent 行与 config_json	名称、提示、默认模型、能力开关	存储表示不等于生效表示
后端解析	workspace_dir、backend	文件内容与执行环境分开
Provider 解析	具体模型引用与候选	自动选择与显式选择不同
Connector / Plugin / Skill	工具、MCP 配置、技能目录	只有启用且有效的能力应进入
安全配置	SecurityPolicy、interrupt_on	Agent 覆盖不应绕过全局约束
Memory 配置	后端与辅助抽取模型	与聊天模型未必相同

源码还根据 HarnessAgentConfig 可用字段进行兼容处理。这提醒我们：宿主与运行库各自发布，安装依赖满足版本下限并不等于所有可选行为都完全相同。分析宿主代码时不能默认最新库内部契约永远不变。

中间件不是任意排序的清单

宿主装入配额、reasoning、知识搜索提示、浏览器 profile、二进制读取保护、图片材料化、线程产物记录和 octop_ui 大载荷卸载等中间件。外层还有按条件加入的模型重试或安全 failover。

顺序会改变谁看到原始内容、谁看到精简内容。源码注释明确要求 OctopUiOffload 保持在特定内层，使其他中间件观察到一致的瘦身结果。它针对插件 octop_ui envelope，不应被泛化为“所有工具输出统一压缩”。

以访谈附件为例，工作区图片路径需要在模型调用前变成模型可读的内容；文件生成以后，又要把路径关联到当前线程。一个处理输入材料，一个记录输出产物，虽然都涉及文件，却不该合成一个万能文件中间件。

并发有不止一种锁

manager.py 中可以定位按 Agent 的生命周期锁、按 Agent 与 thread 的执行锁、在途调用计数及维护相关保护。它们解决的冲突不同。

研究专家配置重载时，不应同时被另一路启动或停止；同一线程的流式调用与 HITL 恢复也不应任意交叉写状态。记忆精简或恢复则涉及存储维护，不能在仍有写入时随意替换文件。

这些锁是本进程的协调工具，不是跨实例锁。即使两个进程用了相同 agent_id，它们也不会共享 Python 的 asyncio.Lock。

重载并非唯一更新方式

部分变更需要重建运行时，例如某些 Provider 或插件配置；部分启用禁用可热同步，例如技能过滤集合。源码还有“等待中的图刷新”与有界并发重载逻辑。

由此可以得到一个维护原则：每个配置写入接口，都应明确它是仅持久化、原地更新还是重建。若 API 已返回成功而更新在后台执行，界面还需要能表达实际状态与失败信息。

▼ 一个适合重构但不必立即重构的切口

若需要降低 manager.py 的认知负担，可以先把“将已解析输入生成配置”的部分提取成可测试步骤，保留生命周期所有权。这里是演进建议，不是认定文件超过某个行数就必须拆服务。先增加配置组合的契约测试，比引入一个远程装配服务更直接。

源码入口

- `src/octop/infra/agents/manager.py`
- `src/octop/infra/agents/middleware/model_failover.py`
- `src/octop/infra/agents/middleware/octop_ui_offload.py`

动手检查

林岚禁用了某个 Skill，又替换了模型供应商。为什么不能让两个操作统一执行“更新数据库后什么都不做”？

答案提示： 运行对象可能仍持有旧过滤集合、旧模型 factory 或旧工具绑定。需要按变更类别热同步或重建，并明确在途任务使用哪个配置快照。数据库记录正确只是必要条件。

05

身份与会话：四种 ID，四条边界

一位用户可以使用别人的专家

前面的配置装配解决“专家有哪些能力”，还没有解决“当前请求属于谁”。Octop 至少要区分 user_id、agent_id、session_key 和 thread_id。把四者合成一个字符串会让权限和生命周期互相干扰。

标识	回答的问题	典型变化
user_id	当前主体是谁	登录用户改变
agent_id	使用哪位专家	切换专家
session_key	哪个入口会话绑定当前线程	Web、CLI、IM 来源改变
thread_id	哪份持续对话与图状态	新建、切换或分叉线程

Agent 归属是资源关系，thread 归属是对话关系。林岚分享了研究专家，周越可以使用它，但不应因此获得林岚的私有 thread。

先授权专家，再授权线程

api/common/agent.py 将访问与管理分开。所有者、管理员和满足分享范围的用户可以获得不同访问能力；修改专家配置则有 owner 级检查。用户组分享还会进入 expert_visible_to 等可见性逻辑。

聊天准备阶段会再次校验指定线程。以下是 resolve_thread_id 中的核心判断，省略了后续绑定操作：

```
row = thread_registry.get_thread(thread_id)
if row is None or row.agent_id != agent_id:
    raise OctopError(ErrorCode.AGENT_NOT_FOUND, ...)
if row.user_id != user_id:
    raise OctopError(ErrorCode.FORBIDDEN, "thread not owned by user")
```

这里省略号是教学摘录标记，不是可运行代码。第一关防止给 A 专家塞入 B 专家的线程，第二关防止同一共享专家下读取别人的私人线程。只验证 JWT 或只检查 agent_id 都不够。

session_key 是入口绑定，不是授权凭证

Web 默认会话键可表示为：

```
<agent_id>:dashboard:<user_id>:dm
```

CLI 与 IM 有各自的键规则。IM 会把平台会话及私聊/群聊语义纳入区分。thread_id 本身不负责编码来源，来源信息可以单独存入线程记录。

为什么不直接拿 session_key 当 thread_id？因为同一个入口可以切换到新线程，旧线程仍需保留。绑定可变，对话身份稳定。ThreadRegistry 因而负责创建、绑定、归档等关系，而不只是提供一个随机 ID。

任何由客户端带来的 session_key 都只能当作待验证的路由输入，不能因为字符串里包含某个用户编号就当作服务端已证明身份。本书并未穷尽审计所有调用者，这是一条需要持续测试的安全不变量。

用一组具体 ID 走一次

教学例子里，林岚为 user 7，周越为 user 9，研究专家为 research。两人各自使用：

```
research:dashboard:7:dm → thread-L
research:dashboard:9:dm → thread-Z
```

专家共享以后，两条映射仍应独立。林岚新建会话时，自己的入口可改绑 thread-L2；thread-L 作为旧对话保留。周越复制 thread-L 的 ID 发起订阅，服务端应拒绝，而不是因为两人选中同一专家就放行。

再加一个 IM 群聊，它是新的入口语义。要不要和 Web 共用对话，不应靠碰巧相同的字符串决定，需要明确的绑定规则和授权。

资源删除也必须沿标识追踪

删除专家可能涉及线程、checkpoint、记忆、工作区和正在执行的调用。删除一条线程则不等于删除专家的全部长期记忆。manager.py 的线程清理与回收逻辑提示，这些动作已经不是简单的一条 DELETE。

制定数据删除需求时，要列出每种数据归属。否则界面里删除了会话，用户可能以为历史、工具产物、派生记忆和备份都消失了，实际只移除了一个视图对象。

▼ 如何把权限规则变成测试

准备同一所有者、共享用户、未授权用户和管理员四个主体。分别测试专家读取、配置写入、线程读取、WS 订阅与取消。共享用户“可聊天、不可改配置、不可读别人的 thread”必须是三条独立断言。

源码入口

- `src/octop/api/common/agent.py`
- `src/octop/api/routers/chat/turn.py`
- `src/octop/infra/gateway/threads.py`
- `tests/integration/test_boundary_authz.py`

动手检查

周越没有专家所有权，但有分享访问权限。为什么允许他创建自己的对话，却不允许他改这个专家的模型？

答案提示： 运行能力的使用权与资源配置权分开。模型和工具改变会影响专家所有者及其他使用者；聊天则可通过独立线程控制范围。不能把“能访问资源”的布尔值复用成全部管理许可。

06

存储与历史：不要寻找唯一数据库

“聊天丢了”可能有四种含义

林岚刷新页面后看不到最后一段文字。它可能从未被模型成功产出，也可能产出后未持久化，可能已保存但读取失败，也可能已读出却被前端状态合并覆盖。单看截图，无法判断数据库是不是坏了。

Octop 的状态分散在不同用途的存储中。理解这张表，比先背数据库表名更有用。

类别	保存什么	不应替代什么
控制面	用户、专家、Provider、通道、cron、设置、权限等	图执行现场
checkpoint	线程执行状态与消息上下文	面向用户的完整历史契约
展示历史 / 轨迹	消息投影、工具过程、回合状态	外部动作的最终业务凭证
Agent Memory	提炼、召回和维护的长期信息	原始材料与当前事实核验
工作区	提示文件、技能、输入附件和输出产物	控制面所有配置
知识索引	文档切片与 embedding	聊天状态与权限来源

SQLite 与 PostgreSQL：接口相近，约束仍在

DatabasePool 协议提供 connect、transaction 和 close。默认 SqlitePool 使用一条共享连接，以 RLock 协调本进程访问，开启外键与 WAL。事务使用 BEGIN IMMEDIATE，提前取得写入相关锁，降低先读后写时遇到快照冲突的风险。

PostgresPool 在边界提供兼容层，仓储里的问号占位符转换成 psycopg 形式。当前 qmark_to_pyformat 是简单替换，源码说明它不理解 SQL 字符串字面量。它适用于受控仓储 SQL，不是一个通用 SQL 方言转换器。

迁移文件有 SQLite 与 PostgreSQL 两套。控制面不负责安装 vector 扩展，记忆的 DDL 也不应混入控制面迁移。ADR 002 特别强调单个活跃 Octop 写入者和不提供 SQLite 到 PG 的自动数据迁移承诺。

把 checkpoint 当历史库会怎样

运行图保存状态，是为了继续执行、上下文恢复与中断处理。聊天界面希望获得稳定分页、说话人、工具展示、错误状态和导出。这两个读写模式不同，所以项目增加了消息投影及独立历史服务。

当上下文做压缩时，后续模型不一定再读取之前全部内容；用户仍可能需要看原始对话。反过来，历史里存在一段可读文本，也不能证明图能从那里恢复到同一个工具等待点。

因此“备份了聊天导出 JSON”不等于备份可恢复的运行环境；“保留了 checkpoint”也不自动证明用户历史展示完整。

可选 v2 历史为何值得单独看

快照包含默认关闭的 history_v2。它在本地 history_v2.sqlite 中按段管理历史，用 documents 与 bodies 保存结构和可复用正文。旧段并不批量迁移，新旧格式经兼容读接口统一展示。启用后需要保留对应 required 标记。

它的可靠性取舍很明确：需要保存的流式内容先等待事务提交，再转发给客户端。于是有两种可解释的崩溃窗口：

> 提交前退出：这条路径不会先把未提交内容发给前端
>
> 提交后、发送前退出：历史已有，前端可能还没看到

这不是外部工具精确一次执行保证，也不涵盖上游从未发给采集器的事件。逐事件提交还会增加事务次数及延迟，可靠性收益有成本。

v2 只支持 SQLite 控制面。存在归档文件或标记时，不能假定把开关关掉就能删除文件，或直接退到不认识新格式的旧版本。

三个数据库不等于一个事务

一轮任务可能写控制面线程信息、历史归档和 Agent checkpoint。这些存储没有因为处于同一进程就自动形成跨库原子事务。

docs/versioned-history.md 明确说明运行中多库备份不保证同一瞬间，并限制涉及 v2 的现有在线恢复与运行时换库。更稳妥的恢复实验，是停机获取完整目录副本，用兼容版本在规定路径恢复，再验证各层引用。

不要只复制一个 octop.db 就宣布已备份全部对话。也不要把不同时间点的主库、归档和工作区随意拼成一个“最新备份”。

▼ 数据库表阅读顺序

先看 users、agents、sessions、threads 的身份关系，再看 thread_messages 与 trajectory_events 的投影；随后对照 providers、channels、cron_jobs 和 settings。用户组、角色与分享范围还要看后续迁移。001_initial.sql 是初始结构，不是完整最终 schema。

源码入口

- `docs/adr/002-database-backends.md`
- `src/octop/infra/db/pool.py`
- `docs/versioned-history.md`
- `src/octop/infra/history/service.py`
- `src/octop/infra/history/recorder.py`
- `src/octop/infra/db/migrations/001_initial.sql`

动手检查

工作室决定迁移到 PostgreSQL，并同时开启 history_v2，把全部聊天记录放到远程数据库。这个方案哪里不成立?

答案提示： v2 在该快照下要求 SQLite 控制面，且实际使用本地独立归档；知识索引也仍有本地 sidecar。要按数据类别评估迁移，不能用一个控制面选项推导全平台存储行为。

07

工作区后端：远程文件不等于远程执行

附件不是一条预览 URL

林岚上传访谈时，浏览器可能获得一个预览链接和一个工作区路径。后续聊天准备会优先根据 workspace_path 形成附件内容；只带预览 URL 的块不能等价为已经入库、可被工具读取的附件。

这背后的问题是权威位置。浏览器看得到一个地址，不说明模型工具能读取它，也不说明它属于当前专家。先落到受控 inbound，再传递工作区引用，才能让输入、读取与权限检查对齐。

BackendWorkspace 隔离的是内容访问

Octop 支持本地文件系统、对象存储、PostgreSQL 文件后端，以及 Docker、OpenSandbox 等后端形式。adapter.py 将控制面后端行转换为 harness 的 backend spec；resolver 再结合 Agent 配置选择实际后端。

这不表示这些后端都有完全相同的执行与文件语义。把文件存进 S3 解决的是存取位置；在 Docker 中执行命令涉及进程、挂载与权限。它们可能出现在同一配置体系，仍必须分别分析。

项目的 agent-backend-file-io 文档要求 Agent 内容文件通过工作区接口访问，而不是直接 Path.read_text。假如编辑专家改用 COS，而 API 仍直接读本机同名 SOUL.md，就会出现“页面说已修改，实际模型读旧内容”的分叉。

两类路径必须区分

```
内容路径：SOUL.md、skills/、inbound/、生成文件
    → 走 BackendWorkspace 的内容访问

本地运行材料：部分 checkpoint、harness 管理的状态与回退文件
    → 按运行库及所选后端的具体约定处理
```

docs/architecture.md 描述了本地运行文件的默认安排；ADR 002 又说明 PostgreSQL memory 场景可复用相关 checkpoint 存储。两者需要组合阅读，不能得出“checkpoint 永远是本地文件”这种绝对结论。

本地 workspace_dir 有时仍承担运行锚点，即使某些内容已搬到远程后端。看到本地目录存在，不说明所有内容都以本地副本为权威。

用一次提示文件修改看差异

教学流程是：界面提交新的专家准则，服务端验证专家配置权限，取得该专家的工作区接口，写入准则文件，再根据能力选择同步或重载。

错误实现是直接拼接本地路径并写入。它在默认 filesystem 下看似正常，换到远程后端才暴露。另一个错误是先对前端返回“保存成功”，把实际后端写入留给无人跟踪的后台任务。

远程后端失败需要成为可见结果。否则林岚会以为专家已经改为“外发前审批”，实际执行仍使用旧规则。这已经从存储问题变成安全预期问题。

后端替换需要契约测试

新增后端时至少核对文本和二进制、目录列举、相对路径、覆盖写、缺失文件、路径越界、下载和失败传播。对象存储没有天然 POSIX rename 语义；远程延迟也会改变读取大量小文件的代价。

执行能力还要单列：shell 在哪里运行？浏览器是谁的进程？工作区文件如何进入沙箱？网络请求是否受同样约束？不要用“支持 Docker 后端”代替对所有工具执行环境的检查。

这些是接入验收建议，不代表本书对每一种后端执行了兼容性测试。

▼ 与 SAM 工程分层的一个对照

支付系列把渠道差异留在渠道适配层，避免编排层直接面对每家接口。工作区后端也有相似的边界价值：上层提出读写内容的需求，不应到处重写 COS、S3 或本地路径细节。相似的是隔离差异的方法，不是要求照搬 Java 的模块结构。

源码入口

- `docs/agent-backend-file-io.md`
- `src/octop/infra/backend/adapter.py`
- `src/octop/infra/backend/resolver.py`
- `src/octop/api/routers/chat/turn.py`

动手检查

一段新代码用 Path(workspace_dir) / "inbound" 找附件，本地测试通过。为什么仍不能认为符合后端抽象？

答案提示： 它绕过内容后端，隐含附件必在本机该路径。应查看调用者需要的是后端内容引用，还是已按契约材料化的本地临时文件；只有后者才可按明确生命周期使用本地路径。

08

一次对话：从 WebSocket 到模型再回来

先把入口和执行者分开

前面已经有用户、专家、线程和存储，现在追一次真实入口。林岚点击发送后，浏览器连接 /api/agents/{agent_id}/chat/ws。这个连接负责接收 user_turn、订阅、取消与控制消息，不直接承担全部模型循环。

服务端先解析 token、检查专家访问，再注册连接。用户提交的消息经过结构校验和 prepare_dashboard_turn，解析线程、模型选择、MCP、Skill 与附件。接着生成 InboundMessage，交给 ChannelManager 排队。

这里是进程内消息调度，不是引入了 Redis 或外部 MQ。ADR 001 的“无外部队列”与内部 enqueue 不矛盾。

请求被翻译成共同语言

Web 表单与 IM 消息形态不同。统一的 InboundMessage 把差异收敛到内容、主体、通道、tenant_id 和 metadata。此路径中 tenant_id 用来选 Agent，不应直接理解为拥有完整 SaaS 语义的租户隔离层。

GlobalProcessor.iter_turn_chunks 为使用原生 chunk 的入口处理请求；IM 路径则还会经 project_stream 转成 MessageEvent。模型不需要自己理解飞书消息格式或浏览器组件状态。

```
user_turn
 → 校验用户、专家、线程、附件
 → InboundMessage
 → ChannelManager.enqueue
 → GlobalProcessor
 → AgentManager.stream
 → HarnessAgentManager.stream
 → 模型及工具循环
 → chunk 记录、补充、投递
```

这是控制流。存储提交、工具执行和网络发送不共享一个大事务。

一个关键顺序：先订阅，再让出控制权

ws.py 有一段容易被当成细节的逻辑：线程确定后，先 hub.subscribe，再等待 steer 接纳或其他后续处理。因为 await 会把执行权交出去，另一个运行中的 turn 可能马上产生消息。

如果先 await 再订阅，消息可能已经向房间推送，而此连接尚未加入，最初几帧就丢了。这个问题和模型速度无关，是异步调度的时序问题。

接下来再判断本轮是否作为 steer 进入正在运行的任务，或者作为新 InboundMessage 入队。不是每次用户追加文字都一定启动一个全新独立模型循环。

主处理器在模型前后做什么

普通请求进入运行时之前，要处理 slash、HITL 待答、线程绑定、模型和能力配置。并不是所有输入都会到 LLM；如某个管理命令已由宿主解释，就可以直接产生响应。

运行过程中，宿主跟踪 usage、历史与轨迹。以下是当前源码中的关键顺序摘录：

```
usage_tracker.observe(chunk)
history_tracker.observe(chunk)
await flush_tracker(history_tracker)
```

这几行之后还有轨迹记录、HITL 注册、工具结果增强和对外 yield。flush_tracker 是否形成具体提交，取决于选中的 tracker 与历史格式，不能把这三行解释成所有模式都具有相同持久化保证。v2 的提交先于发送约束见存储章。

工具结果还可能关联工作区产物并转换成附件帧，且做去重。否则一次重放的工具结果可能让界面反复出现同一个下载文件。

断线与取消是两件事

本地聊天路径的 finally 注释明确：连接断开不应取消在途 turn。服务端注销连接，任务仍可继续。浏览器回来可以重新订阅，继续接收后续帧。

这叫弱流式恢复，不等于从任意帧号完整重放断线期间每一个 token。缺口要依赖历史读取与状态协调弥补，不能只凭“WS 重连成功”宣布无损。

显式 cancel 则校验线程后，标记 steer 取消、丢弃待接纳内容并向 Agent 注册表发取消信号。取消是请求，不是外部副作用的回滚：已经发出的邮件、已经写出的文件不因停止流式回答而自动撤销。

模型失败不应变成一条成功回答

processor 使用 stream_ok、异常记录和 finally 清理区分完成与未完成。失败时产生 error 并保存不完整回合，而不是只把错误文案当作普通 AI 答案混过去。

模型 failover 也有边界：SafeModelFailoverMiddleware 只在自动路由及适合的失败类型下尝试候选，同时检查多模态需要、健康状态和上下文容量。它不是“任何异常都换模型再执行整个任务”。

▼ 排查一次“没收到回复”

依次问：WS 是否认证通过？线程是否正确订阅？消息是否入队？处理器是否启动？模型是否返回 chunk？历史写入是否成功？投递到哪个 thread？前端是否拒绝了错误线程的帧？每个问题对应不同文件，不要从 UI 没字直接跳到更换模型。

源码入口

- `src/octop/api/routers/chat/ws.py`
- `src/octop/api/routers/chat/turn.py`
- `src/octop/infra/gateway/process/processor.py`
- `src/octop/infra/gateway/process/stream_project.py`
- `src/octop/infra/agents/middleware/model_failover.py`

动手检查

林岚在生成过程中关掉页面。过一分钟回来，历史显示最终提纲。能据此证明浏览器离线时所有 token 都逐个重放了吗？

答案提示： 不能。它只证明最终可读结果可能已持久化并被重新获取。服务端继续执行、后续订阅和历史补齐是三件事，逐帧重放需要额外的序号、保留和确认协议。

09

记忆与知识库：两条不同的检索路径

昨天的约定与今天的资料

林岚说过“优先给中文资料”，这可能适合成为 Agent 的长期记忆。上传的出版规范则是一份有版本和来源的文档，适合进入知识库。两者都影响回答，但保存的单位、权限范围和更新方式不同。

聊天历史记录这次说过什么；checkpoint 支撑图的状态；Memory 组织可复用信息；知识库索引文档证据。它们都可能存文本，不意味着可以合并成一张“聊天信息表”。

Memory 后端由宿主解析

memory_backend_from_agent_config 读取 Agent 配置。没有显式指定后端时，SQLite 控制面保留运行库的默认；PostgreSQL 控制面则默认向 Memory 传同一 DSN，并按 Agent 命名空间区分。显式 sqlite 覆盖仍然存在。

这是存储选择，不是授权策略。让多个专家复用一个 DSN，并不等于它们共享记忆；反过来，分开文件也不能代替应用层访问控制。

源码还给记忆辅助抽取模型做解析与兼容处理。日常聊天用的默认模型和记忆维护模型可能不同。因此“聊天正常、记忆提炼失败”并不矛盾，应检查各自模型配置及可用性。

本书不展开外部 octop-memory 的全部提取与召回算法，避免把宿主参数名字当成内部算法证明。

知识库索引没有依赖大型向量服务

KnowledgeIndex 的类注释很直接：每个知识库使用一个本地 SQLite sidecar。chunks 表保存 chunk_id、doc_id、ordinal、text、embedding 和 metadata；embedding 以二进制形式保存。

replace_doc_chunks 在事务中先删除该文档旧片段，再写入新的片段。它处理的是单个索引内的文档替换，不保证“上传源文件、解析状态、embedding 服务、索引内容”跨组件原子提交。

search 在进程中计算余弦相似度。这样的实现部署简单，但不能根据“支持向量检索”四个字就假设已经用了 ANN 索引或独立向量数据库。数据增长时应测量扫描量、向量维度、内存与延迟。

检索先限制集合，再排序

retrieve_context 的读取流程可以概括为：

```
检查知识能力与 query
  → 为 query 生成 embedding
  → 取得当前主体可见知识库
  → 与本轮选中的知识库取交集
  → 在 ready 文档范围内筛选命中
  → 汇总并按 score 排序
  → 截取 k 个候选
  → 在字符预算内拼接上下文与引用
```

当前默认 k 为 8，正文字符预算为 6000。这是快照中的实现常量，不是所有模型的通用最佳参数，也不是 token 预算。中文、英文与代码的字符到 token 比例不同。

权限应在候选集合形成之前起作用，不能先搜出别人材料，再指望提示词让模型别引用。ready 文档限制则避免把未成功完成索引的材料误当可用证据。

一个可见的降级点

retrieve_context 把同步检索放到 executor，外层捕获失败后记日志并返回空字符串。收益是知识检索失败不必让整个聊天崩溃；代价是用户可能收到没有私有证据支持的普通回答。

对工作室的出版规范查询，这个降级是否可接受取决于产品约束。如果要求“只能按上传规范回答”，检索不可用时应显式提示而不是静默继续。这是演进建议，不代表当前所有入口已实现严格失败关闭。

引用标记也只说明来源关联，不自动保证答案完全受引用支持。验收应检查最终模型输入、选中的片段和实际回答，而不只看界面有没有一个引用徽标。

▼ 一个有用的对照实验

准备两份虚构规范，分别可被林岚和周越读取。让两人提出相同关键词。记录可见集合、命中片段、注入正文及回答引用。这样能区分授权错误、召回错误、注入缺失和生成错误；只测“模型答得像不像”无法定位问题。

源码入口

- `src/octop/infra/agents/memory/backend.py`
- `src/octop/infra/knowledge/index.py`
- `src/octop/infra/knowledge/retrieve.py`
- `src/octop/infra/knowledge/jobs.py`

动手检查

控制面切到 PostgreSQL 后，本地 knowledge 目录能不能直接删除，等系统从 PG 里自动恢复全部向量?

答案提示： 不能这样推断。知识索引在该快照里是本地 sidecar，需检查原始文档与索引重建路径。控制面远程化没有自动迁走所有数据。

10

工具与扩展：Skill、Plugin、Connector 和 MCP

四种扩展不是四个同义词

工作室想让专家按固定规范写稿，想加一个稿件状态面板，还想连腾讯文档。这三种需求不该都用“安装插件”笼统处理。

机制	主要解决什么	典型产物
Skill	给 Agent 可发现、可读取的操作知识	SKILL.md 与配套文件
Plugin	宿主/运行库可加载的程序扩展	工具、中间件、通道或 UI 协议载荷
Connector	把某用户的外部服务账户配置接成能力	凭证、实例配置、可用工具
MCP	一套工具发现和调用的互联协议	server 配置、工具 schema 与调用

技能可以指导模型调用工具，但不能代替服务端鉴权。MCP 能传递工具请求，也不会自动为每个工具赋予最小权限。

从用户配置到本轮工具

装配时，AgentManager 从 ConnectorService 构造用户对应的 MCP 配置，结合内置、知识库、移动端和插件工具形成运行时能力。部分能力可以在当轮选择中继续收窄。

需要区分三个集合：系统安装了什么、这位用户授权了什么、这位专家这一轮可用什么。如果插件已安装但未启用，不应仅因它出现在磁盘上就进入每个专家的工具列表。

PluginManager 启动时播种 bundled 插件并加载已安装插件。官方架构文档指出 bundled 默认关闭。插件代码在宿主边界执行，其风险不能与“读取一段技能文本”混为一谈。

Connector 模式决定实际调用位置

catalog.json 是内置连接器元数据源。不同模式有不同路径：

- remote：运行库连接供应商 MCP 地址。
- gateway：使用进程内 Python 适配器，配置中可能只是名称占位。
- internal：Octop 托管 /api/internal/mcp，运行库通过 HTTP 加载。
- 目录中也有 stdio 类型接入，应按相应配置和调用器理解。

同样叫“连接器”，有的走网络，有的进程内执行，还有的启动子进程。超时、日志、凭证传播和资源释放因而不同。新增服务时应先确定调用位置，再设计配置页面。

大型 UI 载荷为什么要离开模型上下文

插件可能返回适合界面渲染的 octop_ui 内容。让模型反复看到一份大型 UI 数据会浪费上下文；只保留一个空壳又会让界面无法显示。

OctopUiOffloadMiddleware 的任务就是处理这类 envelope，使模型侧和 UI 侧按各自需要消费。它的作用范围是特定插件载荷，不是任意工具结果的通用对象存储方案。相关文档独立说明协议，避免前端、工具和中间件各猜一套字段。

这是一个典型产品化问题：运行库只要能返回工具结果，产品还必须考虑结果的体积、展示与后续上下文预算。

一次外部写操作需要几道门

假设模型准备把提纲写进腾讯文档。至少需要确认当前用户的连接器实例有效，工具在当前专家上启用，参数符合 schema，工具策略允许或触发审批，真正调用供应商后得到可核验结果。

供应商请求超时尤其危险：可能从未写入，也可能已写入但返回丢了。模型重试不是业务幂等。若工具支持幂等键、查询已创建文档或可对账操作记录，应在工具边界使用，不能让模型“再试一下”承担去重职责。

这与 SAM 系列回调幂等的讲解思路相通：先找重复从哪里来，再为具体动作选稳定标识。它不意味着 Octop 已内置支付式三层幂等，或每个 MCP server 都支持同一保证。

▼ 新增连接器时的最小检查表

核对凭证属于谁、如何加密、授权可否撤销、工具命名是否冲突、schema 是否兼容、读写是否区分、超时能否判定结果、日志是否含秘密。探测成功只证明一条接入路径可用，不证明所有业务动作安全。

源码入口

- `src/octop/infra/agents/manager.py`
- `src/octop/infra/agents/plugins/manager.py`
- `src/octop/infra/connectors/builder.py`
- `src/octop/infra/connectors/catalog.json`
- `docs/octop-ui-payload-offload.md`
- `plugins/README_CN.md`

动手检查

一份 Skill 写着“只有管理员可以外发”。普通用户能否因此被服务端可靠阻止调用发送工具?

答案提示： 不能。模型提示是行为指导，不是服务端授权。必须在工具调用边界根据认证主体与策略校验，必要时进入 HITL；不能把用户可修改的文本当安全策略源。

11

多 Agent：房间共享，状态分离

把合作拆成编排与执行

现在让研究专家整理背景，法务专家核查风险，主持人汇总结果。若只把三个人的消息拼在同一份上下文里，谁负责工具、记忆与取消会很快混乱。

Octop 的团队主持人是 agents.kind=team 的特殊专家，成员仍是独立专家，可以参与多个团队。团队编制在主持人工作区的 .octop/manifest.json 中，数据库 kind 用于索引。这里没有一张被本书假想出来的独立成员主表。

普通专家和团队主持人使用同一个 Agent 生命周期体系，但工具权限与调用模式不同。主持人主要协调，不承担完整文件、浏览器和 MCP 执行能力。

一个房间，三份执行状态

```
房间 conversation_id = 主持人 thread_id
  主持人 checkpoint: conversation_id
  研究成员 checkpoint: conversation_id~research
  法务成员 checkpoint: conversation_id~legal
```

```
各专家 → 各自工作区 / 记忆
成员流 → 房间转播，带说话人标记
```

用户看见同一条时间线，不表示多个 Agent 写同一个 checkpoint。展示共享与状态共享是两回事。

成员得到的是主持人改写后的任务说明，群聊历史作为背景；团队准则与记忆在主持人侧，派工时需要明确交付。模型不能因为“加入团队”就自动读取所有同事的私有工作区。

派工、回叫与收尾

当前专家团队文档描述：普通专家侧以同步互问为主；主持人异步派工后立即返回，成员完成后由平台回叫主持人。成员能同步询问同事，但不再异步往群里拉人。

异步调用需要保留 source_agent_id、source_thread_id、目标与任务标识，才能把后续结果送回发起位置。否则成员几分钟后完成时，平台不知道回到哪个房间，也不知道还应不应该唤醒主持人。

当前文档说明 inbox 按 callee 协调，同一成员串行、不同成员可并行，主持人回叫按源 thread 串行。早期 mailbox 方案写过单 worker 串行，不能把旧方案当当前宿主契约。具体库内调度器仍需在固定 octop-harness 版本中独立复核。

为什么要按 job_id 幂等记账

团队有在途派工时，不允许随意移除成员或删除对应专家。
TeamJobTracker 为这项约束记录 busy 状态。

下面是 begin 的真实核心片段：

```
if job_id:
    if job_id in self._jobs:
        return
    self._jobs[job_id] = key
    return
```

同一个 job 重复通知开始，不应把繁忙计数加两次。结束时按 job_id 移除，也比“每条结束事件都减一”更稳定。这里幂等的是在途记账，不是成员执行的全部外部副作用。

TeamJobTracker 使用进程内 dict 和 Lock，重启丢失，源码注释明确它不能为重启前残留任务提供 fencing。不能据此宣称有持久任务队列或跨进程安全。

点停止，究竟停止了谁

团队文档明确：Dashboard 取消只取消主持人当前 stream；已排队或正在运行的成员不因此取消。成员仍可能完成并回叫。

对林岚来说，这可能表现为“我明明停止了，研究专家还在回话”。它不必然是实现错误，而可能是当前取消语义。界面与文档应清晰表达停止本轮和停止整组任务的差别。

如果工作室要求“一键取消全部，并阻止后续外发”，就需要扩展任务树取消、回调抑制与工具边界检查。只在前端忽略成员消息，不会停止实际工作。

团队与子 Agent 也不要混淆

本仓库既有 subagent 目录，也有完整专家团队。一个运行时里的子任务，与拥有独立身份、工作区、分享和会话的专家成员，生命周期不完全相同。

选哪一种应看需要共享什么、隔离什么、结果回到哪里，而不是看“多 Agent”标签是否相同。能用一个临时子任务解决的处理，不一定需要创建一位长期专家。

▼ 把支付回调的阅读方法用在团队协作

SAM 编排章节会追踪“哪个子结果到了，什么时候合并”。阅读团队也可以这样做：谁派工、谁收结果、哪个 job 已结束、由谁判断收工。但团队总结不是资金事务，不要套用全成功或全回滚的承诺。

源码入口

- `docs/expert-teams.md`
- `docs/agent-interop-mailbox.md`
- `src/octop/infra/agents/teams/jobs.py`
- `src/octop/infra/agents/teams/team_manager.py`
- `src/octop/infra/agents/teams/service.py`

动手检查

主持人停止后，研究专家创建了文件并回叫。为了做到“界面上看起来已停止”，直接丢弃所有后续帧是否足够?

答案提示： 不足。只是隐藏信息，既没取消工具，也没协调历史和任务状态。应先定义取消的范围与已发生动作的处理方式，再决定哪些回调可以继续、哪些必须标记取消。

12

安全边界：能聊天不等于能管理

认证只回答第一个问题

林岚已经登录，是否就能下载任意工作区文件、取消任意线程或调用任意工具？显然不能。认证解决“是谁”，资源授权解决“能操作哪个对象”，工具策略解决“这次动作是否允许”，执行环境解决“动作能触及哪里”。

JWT 中间件为 /api/ 路径设有明确豁免，并把已认证用户缓存到 request.state。WebSocket 路由还要自己处理相应认证，因为 HTTP 中间件的存在不自动等价为所有 WS 都被保护。

首次安装还有 setup_lockdown：没有用户时，非设置与健康等允许入口返回 setup_required。这与已经登录后的资源校验是不同阶段。

可见、可用、可写分开检查

api/common/agent.py 的 require_agent_row 处理所有者、共享和管理员相关检查；require_agent_owner_row 再要求管理权。聊天订阅、取消与 resolve_thread_id 又校验 thread 的用户和 Agent。

沿这条调用链检查，比泛泛说“有 JWT 多用户隔离”更可靠。因为同一个专家可能被分享，单凭 agents.user_id 过滤无法描述所有合法访问；而允许访问共享专家也不该放开所有配置和私有线程。

用户组、角色、资源策略分布在 users 与 repos 中。架构书可以解释边界，但不能凭看到这些文件就给出“通过安全审计”的结论。

HITL 保存的是一项待决策动作

危险工具需要人工审批时，运行图暂停。宿主把请求与用户、专家、线程、会话关联，生成 pending 标识；用户的批准或拒绝通过对应恢复路径传回运行时。

审批不是“模型问一句，你回答可以”。需要确认回答针对哪项动作、是否仍有效、是否属于当前主体、是否已经处理。项目有 HitlChannelCoordinator、pending store 与会话策略，用来协调 UI、CLI 和渠道差异。

运行时恢复与普通流式调用也需要线程级执行保护，避免同一等待点被并发恢复。本书未对所有审批竞态执行测试，因此这里描述的是机制与检查入口，不承诺所有工具端到端精确一次。

一条审批链的反例

研究专家准备发送带附件的邮件，界面展示收件人 A；用户批准前，模型或另一路请求把待发送对象改成 B。如果批准仅绑定“发邮件”这个工具名，而没有绑定具体参数和等待点，批准 A 就可能被误用来发送 B。

验收应检查审批参数、pending_id、thread_id 和实际执行内容的一致性。工具参数修改后，是重新审批还是接受编辑，需要清楚契约。通用聊天文本不能成为任意工具的授权凭证。

已经执行的邮件也不会因之后取消 turn 而自动撤回。审批控制未来执行，取消结束计算，补偿处理既有副作用，三件事要分开。

Shell、文件、网络与秘密是不同面

工具护栏管理命令规则；工作区与文件系统保护约束路径；Docker、OpenSandbox 或宿主隔离机制决定进程环境；ssrf_guard 等网络检查涉及请求目标；Fernet 相关代码用于凭证封装。

这些机制可以互补，不能互相替代。限制工作区路径不保证所有网络目标安全；加密数据库字段也不防止运行时日志泄漏。server.py 在普通日志级别降低 httpx/httpcore 输出，理由正是完整 URL 可能含 query secret。

实际部署应使用受信任插件、最小权限凭证和清晰的运行账号。自托管表示控制部署位置，不表示模型供应商、连接器或远程工具永远不接收数据。

Bridge 会扩大需要审视的信任面

跨实例请求经过隧道进入专家所在侧。必须同时确认连接所有者、对端登录身份、可转发路径及最终资源权限。本地 user 7 和远端 user 7 不是同一个人，不能靠整数相等进行身份映射。

当前 tunnel_policy.py 有路径与方法约束。它比“通用 HTTP 隧道”这个协议能力更窄，且与前端远程能力清单可能不同。评审时应检查实际 allowlist，不按规划文档假定所有管理路径都允许或都禁止。

▼ 最小负面测试矩阵

用未登录、普通所有者、共享使用者、无权用户和管理员分别访问专家、线程、文件、取消、审批与管理接口。再加过期 pending、重复批准、路径穿越、内网 URL 和错误桥接身份。正向“能聊一句”不能替代这些负面用例。

源码入口

- `src/octop/api/middleware/jwt_auth.py`
- `src/octop/api/middleware/setup_lockdown.py`
- `src/octop/api/common/agent.py`
- `src/octop/infra/gateway/hitl/coordinator.py`
- `src/octop/infra/agents/security/hitl_session.py`
- `src/octop/infra/utils/ssrf_guard.py`
- `src/octop/infra/bridge/tunnel_policy.py`

动手检查

工具配置写入“只读”，但插件在宿主进程中直接调用文件写接口。提示词和 JWT 能否阻止它?

答案提示： 不能据此保证。插件程序处于不同信任层，需检查实际执行权限、加载来源与策略落实位置。模型侧声明只读不等于宿主级沙箱。

13

前端架构：流式对话是一台状态机

页面不是一份 messages 数组

浏览器要同时表达正文增量、思考、工具调用、工具结果、附件、配额、审批暂停、错误、取消和多人说话。简单“收到 token 就拼字符串，收到 done 就解锁”无法覆盖这些状态。

Dashboard 使用 React、TypeScript、Vite、Ant Design 等组件。读聊天架构时，可以沿着三个问题追踪：谁持有流状态，消息属于哪个会话，历史怎样与实时数据合并。

为什么 chatStore 放在组件之外

chatStore.ts 文件开头说明：WebSocket 与累计消息置于模块级 store，在 React 组件卸载、重挂载后仍存在；组件通过快照订阅更新。

这解决一个具体问题：林岚切到设置页，聊天组件卸载，不应让在途任务的客户端状态立即丢失。回到聊天时可接上最新快照。

这里“持久”仅相对于页面组件生命周期，不等于浏览器关闭或服务端重启后的持久存储。真正刷新需要结合服务端历史和 turn_active 等状态恢复。不要把内存 store 的名称解释成磁盘数据库。

消息属于线程，也属于说话人

parseHarnessChunk 将服务端帧转换为前端可理解事件。frameThread 等逻辑判断帧属于哪个线程；团队流还要依据 agent 标记和说话人规则合并气泡。

如果研究成员的 token 到来时，前端把它接到主持人的最后一条气泡里，文字可能完全正确，身份仍是错误的。若又把成员 done 当作整个房间结束，主持人总结尚未出现，输入框就可能进入错误状态。

所以“一个 WS 连接”不等于“一位说话人”或“一轮唯一完成事件”。同房间多任务是前端状态机复杂度的重要来源。

四个恢复判断各有用途

wsResumeGate.ts 明确把弱恢复与滚动/历史交互分开。当前默认最多自动重新订阅 8 次，无 socket 的陈旧流宽限为 20 秒。这是产品启发式，不是网络可用性 SLA。

它还规定：是否探测活动任务，依据本地流状态或服务端 turn_active，而不是从最后一条消息长得像不像“没说完”猜测。长工具调用可能很久没有 token，socket 仍开着，不应只因沉默就强制封口。

后台未聚焦线程也有不同恢复策略，避免大量标签同时重连。一个等待图标看似简单，背后实际包含活跃状态、连接状态与用户焦点三条维度。

历史回填会与实时增量竞争

假设历史接口刚开始读取时包含 20 条消息，网络返回前又收到新的流式内容。若直接 setMessages(history)，就可能覆盖这段新文本。

相反，如果把历史和当前列表无条件拼接，又可能重复展示同一条工具结果。正确做法需要稳定身份、去重、边界或明确的刷新约束。chatStore 中的历史、绘制、团队流、pending alias 与错误测试，正是在约束这些具体情形。

v2 分页要求原样携带 next_cursor，不能根据新消息数量变化用 offset 自行换算。前端分页协议属于历史一致性设计，不是纯 UI 实现细节。

测试要抓事件顺序

前端测试可以依次注入 token、tool_call、断线、恢复、tool_result、done，再检查消息归属、状态和结果。只断言文字出现，无法覆盖这条恢复路径。

也可以测试：审批暂停时刷新历史、成员结束但主持人未结束、历史读取失败但旧内容还在、取消以后收到迟到帧。事件顺序一换，最终静态截图相同，也可能隐藏状态错误。

这些测试可以离线控制输入，不能替代真实浏览器与后端的联调。本书没有执行 Octop Dashboard 的完整测试。

▼ 从文件名回到事实

sseHelpers.ts 这样的历史命名仍被聊天 store 使用，但普通聊天连接实际由 WebSocket 构造。读代码时看连接创建、订阅及帧处理，不因 helper 名称就把主流程写成 SSE。

源码入口

- `dashboard/package.json`
- `dashboard/src/pages/Chat/hooks/chatStore.ts`
- `dashboard/src/pages/Chat/hooks/wsResumeGate.ts`
- `dashboard/src/pages/Chat/hooks/frameThread.ts`
- `dashboard/src/utils/parseHarnessChunk.ts`
- `dashboard/src/pages/Chat/hooks/chatStore.history.test.ts`

动手检查

有人建议“20 秒没有 token 就结束加载，防止界面卡住”。为什么这会误伤正常任务?

答案提示： 可能正在等待工具、思考或审批。需要结合 socket、服务端 turn 状态和恢复策略；宽限逻辑不能把活连接上的安静任务直接视为完成，更不能把超时封口记成业务成功。

14

跨端协作：ACP 与 Bridge 各管什么

都叫协作，执行位置不同

工作室有两种新需求：编辑想在 IDE 里使用 Octop；另一个办公室有独立 Octop，想借用它的研究专家。前者是客户端/Agent 协议集成，后者是实例间桥接。

能力	对端是什么	执行位置
入站 ACP	IDE 或终端客户端	本地独立 OctopServer 进程
出站 ACP	外部编程 Agent runner	外部 runner 所在宿主环境
Bridge	另一台 Octop	专家所在的远端实例

不要把“远端专家”自动塞进本机 Agent 注册表，也不要把 ACP 当成 Octop 的 HTTP 聊天 API。

ACP 是双向的 stdio 集成

docs/acp.md 说明两种方向均使用 stdio JSON-RPC。入站 octop acp 作为 server，供 Zed 等客户端驱动；出站 acp_runner 作为 client，把任务交给外部工具。

入站命令会启动 standalone OctopServer，并非一定连接已经运行的 octop run。这对运维很重要：若多个进程指向相同数据目录，不能默认它们会共享内存中的锁和调度所有权。

出站 runner 配置是按用户共享，启用工具则按 Agent 控制。命令在运行 Octop 的机器上找 PATH、凭证和工作目录，不是在浏览器机器上执行。给浏览器安装一个 CLI，不能使远端服务突然找到它。

外部 runner 的 permission_required 还需要选择精确 option。不能把内部 HITL 的“批准”直接泛化为外部协议任何选项都已同意。

Bridge 不复制专家

远程专家在本地用 bridge:{connection_id}:{remote_agent_id} 表示。浏览器仍请求本机 API，本机识别远端引用后转发；专家运行、线程权威、附件落盘与历史保留在专家所在侧。

```
浏览器
  → 本机 HTTP / chat WS
  → Bridge 连接：HTTP 隧道与 turn 流
  → 远端 API / GlobalProcessor / harness
  → 远端工作区与历史
  ← 响应、chunk 与按需历史
```

本机可以持有连接配置与临时状态，但不因此成为远端对话历史的第二个完整权威。桥断了以后，不应自动启动一个本地同名专家并续写，那会改变数据和工具归属。

两种 WebSocket 不要混

Dashboard 的 WebSocketHub 服务浏览器到本机的订阅；Bridge 有独立的机机长连接，承载隧道与流式多路复用。浏览器不应直接改成访问远端 baseURL，以绕过本机桥接。

通用 HTTP 隧道可承载 method、path、headers、body 等，但产品安全策略不必允许所有路径。实际 tunnel_policy.py 对专家面、只读 composer、浏览器 viewer 等分别约束，未知路径默认拒绝。

docs/bridge.md 的某些模块清单仍是早期规划，当前实际文件包括 manager.py、transport.py、http_tunnel.py、peer_auth.py 和 peer_turn.py。书中的源码导航使用实际文件，不把规划文件名写成已实现。

同样的用户编号不是同一身份

假设本机林岚是 user 7，远端合作账号是 user 12。隧道在对端用对应登录态执行，资源可见性应由对端用户决定。本机 user_id 不能直接复制成远端 user_id。

共享桥链接的使用者与连接所有者也有不同管理能力。评审要看路径、方法、连接归属及最终授权组合，而不是只判断连接是否已加密。

如果需要防止重复外发，还要跨桥保留动作身份和结果查询机制。Bridge 传输成功不等于工具事务成功；网络超时也不说明远端没有执行。

▼ 当前实现与生产承诺

官方 Bridge 文档仍标注“实现中”。本书只依据快照解释已可定位的结构，没有做跨实例联调、断网压测、版本兼容或安全验收。它适合研究边界，不应直接充当生产可靠性证明。

源码入口

- `docs/acp.md`
- `docs/bridge.md`
- `src/octop/infra/bridge/manager.py`
- `src/octop/infra/bridge/transport.py`
- `src/octop/infra/bridge/http_tunnel.py`
- `src/octop/infra/bridge/tunnel_policy.py`

动手检查

本机和远端各有一位叫 research 的专家。桥连接断开后，自动切成本机 research 是否是透明容灾?

答案提示： 不是。工作区、凭证、知识、记忆、线程和执行环境都可能不同。除非产品定义并实现了可验证的复制与故障转移协议，否则这是换了执行主体。

15

部署与验证：把恢复当成一项功能

一个核心服务，不等于没有其他进程

Octop 的单进程模型描述的是核心宿主。模型服务可以在外部，工具可以拉起子进程，浏览器有自己的进程，Docker 后端也有执行环境。不要把“单进程”写成操作系统里永远只有一个 PID。

wheel 可以带构建后的 Dashboard；开发时 Vite 与 Python 分开运行，发布时静态资源进入 src/octop/dashboard。Docker 则把这些内容和运行环境一起打包，数据目录仍要持久挂载。

desktop/README.md 明确当前桌面壳为 Wails v3，加载打包的 portable runtime，启动 Octop。它与 infra/control/desktop 远程桌面流不是一个模块；仓库残留 Electron 说明也不是当前桌面壳技术栈的证据。

定时任务为何也是主链路的一部分

CronManager 是进程级 APScheduler 管理器，从数据库记录恢复任务。CronJob 把任务交给 CronDeliveryService，并记录运行状态、审计与指标。

task_type 区分直接推文本和运行 Agent 后投递。两者的成本、失败点与权限需求不同。林岚每天收到固定提醒，不一定要花一次模型调用；每天基于新资料生成简报，才需要实际 Agent 处理。

cron 的运行状态 ok 至少表示当前投递流程按其契约成功返回，不自动表示每个外部收件人已经阅读，也不构成所有下游副作用的事务证明。

多进程恢复相同任务时的所有权问题，在这里比“有没有换成 PG”更关键。

备份必须匹配真实数据布局

基础清单应包含配置、主库、秘密材料、Agent 工作区、checkpoint、Memory、知识文件与索引，以及启用时的历史归档和标记。秘密材料要按其恢复依赖安全保存，不能把它们随便打进公开样例。

SQLite 运行中的一致快照不等于复制一个正在写的文件；PostgreSQL 有自己的 pg_dump 路径。ADR 002 不承诺跨引擎恢复。history_v2 的多库协调与路径身份限制则需要额外遵守。

“能导出一份聊天 JSON”和“灾后能恢复整个服务”是两个验收项目。前者帮助核对记录，后者还需要身份、配置、图状态、文件与依赖版本匹配。

按风险层次读测试

测试层	可以验证	不能单独证明
纯逻辑单测	路由、标识、状态合并、参数规则	真实模型与外部服务
API 集成测试	认证、资源边界、WS 与服务生命周期	全部部署环境
PostgreSQL 专项	迁移、仓储、checkpoint 等兼容路径	不依赖本地 sidecar
live 测试	特定凭证与供应商的真实交互	所有供应商与所有权限
恢复演练	指定故障点下的恢复结果	未测试故障和硬件损坏

仓库有 unit 与 integration，并用 marker 控制 live、slow、postgresql 等范围。缺凭证的测试可能跳过。CI 绿灯因此要结合实际执行矩阵理解，不应写成所有真实服务都已验收。

本次只对电子书构建与配套教学模型运行检查，没有运行 Octop 自身测试。

用六次故障推演替代一句“支持高可用”

在隔离测试副本中依次验证：

1. 模型在工具调用前失败，历史和 UI 如何结束。
2. 工具动作成功但返回超时，重试是否重复副作用。

3. 客户端断线后重连，后续帧和历史是否一致。
4. 审批暂停时退出，再启动后待处理动作是否可识别。
5. 写入历史时磁盘满，是否仍向用户宣称成功。
6. 从完整备份恢复，线程、附件、记忆和配置是否能关联。

这套清单没有预设系统应通过所有项目，而是要求把实际结果写下来。有的能力当前不支持，明确限制比伪装成自动恢复更安全。

▼ 上线前先保住哪个指标

先量事件循环延迟、活跃 turn、模型与工具耗时、历史提交延迟、数据库锁等待、重连次数与失败率。注册用户数不能代表同时执行压力，模型 token 速率也不能替代整体请求尾延迟。

源码入口

- `desktop/README.md`
- `docker/docker-compose.yml`
- `Makefile`
- `.github/workflows/ci.yml`
- `src/octop/infra/cron/manager.py`
- `src/octop/infra/cron/job.py`
- `docs/versioned-history.md`

动手检查

工作室每晚备份 octop.db，其他目录不备份。某天服务器故障，恢复后用户和专家列表还在，但附件和记忆缺失。这能算备份成功吗？

答案提示： 只能说部分控制面恢复了，业务恢复目标未满足。应先按数据类别定义 RPO/RTO 与恢复验收，再制定完整备份方案；不能用主库文件存在代替可用性检查。

16

架构评价：何时保留单体，何时拆分

回到开篇的任务

现在可以沿完整链路复述林岚的请求：认证用户选择一位可访问的专家，当前入口绑定自己的线程；附件成为工作区引用，知识检索在可见集合里找证据；宿主组装模型与工具能力，运行库执行；高风险动作进入审批；成员专家各自工作后向房间回传；流式帧、历史、checkpoint 与长期记忆按不同职责保存。

沿着这条链路，可以逐处核对不同职责的交接：身份有没有传对，状态有没有保存，审批有没有挡住外部动作。这些交接构成了模型 SDK 之外的宿主工作。

最值得借鉴的四个选择

第一，围绕自托管约束保留单进程核心。部署简单是面向目标用户的产品价值，不是必须被微服务纠正的技术落后。

第二，让多入口共用运行时能力。Web、CLI、IM、cron 尽可能回到共同消息与处理边界，减少各自维护一套提示、历史和审批逻辑的风险。

第三，工作区后端与控制面分开。配置存在哪个数据库，不应决定每一份专家文件只能放在哪里。这个分离给对象存储和沙箱适配留下空间。

第四，团队共享展示但隔离状态。成员不写同一份 checkpoint，能保留各自工作区与记忆，也让说话人和任务归属有明确表达。

这些是架构分析，不是基准测试结论。

当前复杂度聚集在哪里

AgentManager 与 GlobalProcessor 承担大量配置与生命周期职责，前端 chatStore 也很大。大文件不是自动的缺陷，却说明变更需要同时理解多个契约：普通聊天、团队、HITL、历史、远端桥接与配置兼容。

文档也存在可观察的漂移：README 与当前全局结构不一致，旧 inbox 方案与新团队契约并存，Bridge 部分规划文件名已变化。系统演进时，应把“版本、事实来源与非目标”写在文档里，避免旧设计被误当现行实现。

存储则呈现多层化。它提高局部能力，但会把一致性和恢复问题交给宿主。尤其是新历史归档与旧 checkpoint 并存时，必须明确哪些状态可恢复、哪些只是可展示。

治理顺序比拆分名单重要

建议先做三件小事：

改造	解决的压力	验收方式
固化请求与事件契约	多入口、团队与前端解释不一致	相同事件轨迹在多个入口断言一致
分开配置解析与生命周期	改一个能力牵动全部装配	已解析输入到有效配置的快照测试
建立恢复与负面测试矩阵	正常路径可用但故障语义含糊	故障点、落盘状态与外部动作记录

然后再根据测量决定隔离 CPU 任务、改善索引、减少数据库阻塞或拆分 worker。不要在没有瓶颈证据时，先引入消息队列和分布式事务，反而破坏项目最重要的安装优势。

什么时候才值得进程外队列

若出现大量长任务、必须跨重启保留派工、需要独立扩容执行器，或故障必须隔离到不同执行单元，进程外队列才有明确收益。

但队列并非多加一个组件就结束。至少要设计持久 job ID、任务认领、租约与 fencing、重试分类、幂等副作用、取消传播、结果归档、WS 路由和配置快照。PG 只是其中可能使用的基础设施，不是这些契约的替代品。

第一版可以只把昂贵文档解析移到独立受控执行器，而不拆用户、专家、权限等所有模块。边界应跟随负载和所有权，不跟随文件夹数量。

从本项目迁移到自己的系统

如果你要做小团队 Agent 平台，先保留三个不变量：身份来自认证而不是模型，外部动作在工具边界授权，恢复能力按具体状态和故障点描述。再考虑功能堆叠。

不必完整复制 MBTI、市场、远程桌面或全部渠道。先跑通一次任务的来源、能力、状态、审批与归档，就能建立后续扩展需要的骨架。SAM 系列从需求推进到实现的组织方式适合用来检查这条骨架，但真实技术选择仍由你的业务约束决定。

▼ 一句话的结论及其适用范围

Octop 是围绕自托管体验构建的 Agent 产品宿主，其优势在多入口、多能力的统一装配，主要演进压力在运行时编排、状态一致性与故障边界。该判断只针对本书固定快照，不代表对后续版本的持续评审。

源码入口

- `docs/adr/001-single-process-model.md`
- `docs/adr/002-database-backends.md`
- `src/octop/infra/agents/manager.py`
- `src/octop/infra/gateway/process/processor.py`
- `dashboard/src/pages/Chat/hooks/chatStore.ts`

动手检查

团队提出三个目标：十倍用户、持久任务、模型回答更准确。为什么不能统一归结成“拆成微服务”？

答案提示： 十倍用户要先量活跃负载；持久任务需要任务协议和恢复设计；回答准确性要分材料、召回、提示与模型能力。三个目标的瓶颈可能不在同一层，微服务也不会自动解决它们。

17

代码实战：用最小宿主验证四条边界

不做一个假的 Octop

前面的代码入口分布在宿主、运行库和前端，第一次阅读容易只记住类名。本章把四条边界压成一个标准库程序：共享专家不共享私人线程、入口可以改绑、批准绑定已保存的参数、重复批准不重复创建待投递记录。

程序位于 examples/mini_host.py，测试位于 examples/test_mini_host.py。它不是 Octop API，不调用真实模型，不启动 WebSocket，不发送邮件，也没有实现项目的 Team 或 Memory。用户整数由测试可信地提供，不能用作真实登录方案。

数据模型先于接口

最小模型有五张表：agents 管专家所有者与分享标志；threads 管用户与专家下的私人对话；sessions 管入口到当前线程的绑定；actions 保存待审批内容及状态；outbox 保存批准后的待投递记录。

关键不在表数，而在两个关系：session 指向 thread，不拥有 thread 的全部生命周期；outbox 以 action 为主键，同一个审批动作只能产生一条待投递记录。

所有表都在一个 SQLite 库里，是为了能用一个事务展示本地原子性。它刻意避开 Octop 多层存储和外部网络，不能用这个实验推导生产系统也具有同样的事务范围。

共享专家时仍检查 thread

程序先检查用户是否可用专家，再核对线程里的 user 和 agent。对共享专家的访问允许周越建立自己的线程，却不能让他借用林岚的 thread_id。

```
host.add_agent("research", 7, shared=True)
lin = host.open_thread(7, "research")
zhou = host.open_thread(9, "research")
assert lin != zhou
```

open_thread 的 session key 在服务端根据可信 user 和 agent 生成，不让模型任意选择另一个用户的身份。fresh=True 切换当前入口的绑定，但旧线程仍在，可通过正确身份读取。

这对应第五章的身份边界，而不是模拟完整用户组或管理员模型。

批准时不要重新接受一份待发送正文

propose 先保存收件人与正文，得到 action_id。approve 接收身份、线程与 action_id，不重新接收一个可被替换的 recipient。

这样批准针对的是已保存内容。真实系统仍要考虑审批展示是否与内容一致、动作有效期和参数编辑流程；这个模型只展示最小绑定关系。

程序中的条件更新是：

```
UPDATE actions
SET state='approved'
WHERE id=? AND state='pending'
```

第一次更新影响一行，随后在同一个事务里插入 outbox。重复批准影响零行，不再插入。取消则只允许 pending 进入 cancelled，后来的批准不会将它改成 approved。

本地幂等仍不是外部精确一次

现在假设另加一个投递器，读取 outbox 后向邮件服务发送。邮件已成功发送，但投递器在写回发送状态前崩溃。下次重试时，这条 outbox 仍可能被再次发送。

这个窗口无法由“本地只有一行”自动消除。需要外部供应商幂等键、结果查询、状态协调或明确的补偿机制。若对端不支持可靠去重，产品只能描述实际能保证的语义，而不能宣称邮件精确一次。

这也是本书借鉴 SAM 幂等章节的方法：区分哪一步可能重复，以及每个标识究竟保护哪一个效果。不要把所有重复问题统称为“加一把锁”。

运行和观察

在本书目录运行：

```
python3 examples/mini_host.py
python3 -m unittest discover -s examples -v
```

测试覆盖两用户线程隔离、入口改绑保留旧线程、重复批准、越权批准、取消后批准和关闭数据库再打开后继续审批。默认演示使用内存数据库；重开实验使用临时目录，不操作你的 Octop 数据。

可以先把 approve 中的 state 条件去掉，预测重复批准会怎样；再去掉线程归属检查，观察负面测试是否失败。只在实验副本修改，不要把破坏后的模型接到真实工具。

▼ 实验没有覆盖的能力

没有密码认证、管理员、用户组、HTTP/WS、跨进程任务所有权、外部投递、配额、插件隔离、流式历史和多库恢复。SQLite 重开测试只证明教学模型的待处理记录可重新读取，不证明运行中的模型循环能恢复。

源码入口

- `src/octop/api/routers/chat/turn.py`
- `src/octop/infra/agents/teams/jobs.py`

动手检查

给 outbox 增加一个 sent 字段，然后“发邮件成功后把 sent 设成 1”，能否关闭所有重复发送窗口?

答案提示： 不能。外部成功与本地状态更新之间仍有崩溃窗口。先写 sent 又会在发送前崩溃时漏发。要看对端幂等或查询能力，再设计重试语义，不能用本地字段跨越网络事务边界。

A

附录：源码索引、术语与核验路线

快照信息

项目	值
官方仓库	https://github.com/TencentCloud/Octop
完整提交	`890111b7879665840414ca28ab168b05b0db0921`
包版本	`1.0.3b1`
采集日期	2026-10-27
证据清单	sources/inventory.json，含 64 个文件的固定 URL、SHA-256 与行数
验证范围	静态研究；本书构建和教学示例另有自动检查；未运行 Octop 端到端测试

术语速查

术语	在本书中的意思
宿主	把运行库接成产品的 Octop 服务
控制面	用户、资源、配置与权限等持久状态

术语	在本书中的意思
composition root	集中创建并连接依赖的位置
turn	一次处理回合，不等于一条 token 帧
session_key	某个入口绑定当前线程的会话键
thread_id	持续对话及执行状态的身份
checkpoint	运行图保存与恢复的状态
projection	为某种读取需求形成的消息视图
HITL	工具执行前后的人工参与与恢复机制
inbox	多 Agent 派工和回叫的内部协调入口
outbox	本章教学模型里的待投递记录，不表示已发送
Bridge	Octop 实例间请求与流的桥接
MCP	工具发现与调用协议，不是权限模型
ACP	Agent 与客户端或 runner 的交互协议

先读哪五组材料

1. **起点：docs/architecture.md 与 infra/server.py。** 文档建立地图，代码核对实际单例与装配顺序。
2. **决策：ADR 001 与 ADR 002。** 结合阅读，区分最初的 SQLite 背景和后来加入的 PG 选择。
3. **主链路：chat/ws.py、chat/turn.py 与 GlobalProcessor。** 跟踪身份、绑定、排队、执行、记录和推送。

4. **存储：docs/versioned-history.md、db/pool.py 与 history/。** 先列保证和非目标，再读事务与恢复。
5. **协作：docs/expert-teams.md 与 teams/jobs.py。** 不要将早期 mailbox 设计全文当作新实现清单。

按章节核验源码

以下全部链接固定到本次提交。路径长度较长，网页和 EPUB 中可点击；PDF 可使用链接或从目录定位章节。行数没有写进永久正文，避免把行号当成接口。

00-preface

- `pyproject.toml`
- `README_CN.md`
- `docs/architecture.md`

01-requirements

- `README_CN.md`
- `docs/architecture.md`
- `src/octop/api/common/agent.py`
- `src/octop/api/routers/chat/ws.py`

02-architecture

- `docs/architecture.md`

- docs/adr/001-single-process-model.md
- docs/adr/002-database-backends.md
- src/octop/infra/server.py

03-lifecycle

- src/octop/launch.py
- src/octop/infra/server.py
- src/octop/infra/db/factory.py
- src/octop/infra/db/rebind.py

04-agent-runtime

- src/octop/infra/agents/manager.py
- src/octop/infra/agents/middleware/model_failover.py
- src/octop/infra/agents/middleware/octop_ui_offload.py

05-identity

- src/octop/api/common/agent.py
- src/octop/api/routers/chat/turn.py
- src/octop/infra/gateway/threads.py
- tests/integration/test_boundary_authz.py

06-storage

- docs/adr/002-database-backends.md

- src/octop/infra/db/pool.py
- docs/versioned-history.md
- src/octop/infra/history/service.py
- src/octop/infra/history/recorder.py
- src/octop/infra/db/migrations/001_initial.sql

07-workspace

- docs/agent-backend-file-io.md
- src/octop/infra/backend/adapter.py
- src/octop/infra/backend/resolver.py
- src/octop/api/routers/chat/turn.py

08-turn-path

- src/octop/api/routers/chat/ws.py
- src/octop/api/routers/chat/turn.py
- src/octop/infra/gateway/process/processor.py
- src/octop/infra/gateway/process/stream_project.py
- src/octop/infra/agents/middleware/model_failover.py

09-memory-rag

- src/octop/infra/agents/memory/backend.py
- src/octop/infra/knowledge/index.py
- src/octop/infra/knowledge/retrieve.py

- src/octop/infra/knowledge/jobs.py

10-extensions

- src/octop/infra/agents/manager.py
- src/octop/infra/agents/plugins/manager.py
- src/octop/infra/connectors/builder.py
- src/octop/infra/connectors/catalog.json
- docs/octop-ui-payload-offload.md
- plugins/README_CN.md

11-teams

- docs/expert-teams.md
- docs/agent-interop-mailbox.md
- src/octop/infra/agents/teams/jobs.py
- src/octop/infra/agents/teams/team_manager.py
- src/octop/infra/agents/teams/service.py

12-security

- src/octop/api/middleware/jwt_auth.py
- src/octop/api/middleware/setup_lockdown.py
- src/octop/api/common/agent.py
- src/octop/infra/gateway/hitl/coordinator.py
- src/octop/infra/agents/security/hitl_session.py

- src/octop/infra/utils/ssrf_guard.py
- src/octop/infra/bridge/tunnel_policy.py

13-dashboard

- dashboard/package.json
- dashboard/src/pages/Chat/hooks/chatStore.ts
- dashboard/src/pages/Chat/hooks/wsResumeGate.ts
- dashboard/src/pages/Chat/hooks/frameThread.ts
- dashboard/src/utils/parseHarnessChunk.ts
- dashboard/src/pages/Chat/hooks/chatStore.history.test.ts

14-interop

- docs/acp.md
- docs/bridge.md
- src/octop/infra/bridge/manager.py
- src/octop/infra/bridge/transport.py
- src/octop/infra/bridge/http_tunnel.py
- src/octop/infra/bridge/tunnel_policy.py

15-operations

- desktop/README.md
- docker/docker-compose.yml
- Makefile

- `.github/workflows/ci.yml`
- `src/octop/infra/cron/manager.py`
- `src/octop/infra/cron/job.py`
- `docs/versioned-history.md`

16-tradeoffs

- `docs/adr/001-single-process-model.md`
- `docs/adr/002-database-backends.md`
- `src/octop/infra/agents/manager.py`
- `src/octop/infra/gateway/process/processor.py`
- `dashboard/src/pages/Chat/hooks/chatStore.ts`

17-mini-host

- `src/octop/api/routers/chat/turn.py`
- `src/octop/infra/agents/teams/jobs.py`

参考本地 SAM 系列的方法

本书按用户要求参考本地 learning_sam 的系列博客组织方式。主要对照材料包括支付系列开篇、需求 PRD、技术方案、支付编排与渠道路由，秒杀系列幂等设计，以及 DDD 工程目录划分。它们用于研究如何讲解，不用于证明 Octop 的实现。

借鉴点是需求推导、职责边界、关键代码、完整场景和异常闭环。本文没有复制原博客正文、图片、生产事故和性能指标，也没有把第三方全文归档放入电子书。SAM 材料本身的实现或数字是否可靠不属于本次研究范围。

本地参考入口：

- learning_sam/payment/docs/zsxq-organized/
- learning_sam/seckill/docs/zsxq-organized/
- learning_sam/ddd-practice/docs/zsxq-organized/

本书与原项目的权利说明

Octop 的 LICENSE 为 MIT。本文以自己的分析文字介绍项目，少量摘录保留可核对的源码入口；若再分发上游源码，需遵守其 LICENSE。书稿不是腾讯云官方文档，亦不代表维护者认可。

阅读器布局与部分构建代码沿用用户本地 langchain-langgraph-from-scratch 电子书；正文和教学程序为本次独立编写。未创建远端仓库、提交或公开发布。

▼ 如何复核一句重要判断

找到本章对应源码，核对调用者、参数来源、异常分支与释放路径。涉及外部库时再固定其实际安装版本。看到测试名后读取断言与 skip 条件，不把存在测试文件等同于已经执行通过。

动手检查

“换 PostgreSQL 就能部署多个活跃写实例”这句判断应该去哪两类证据核对?

答案提示： 先读 ADR 002 对单活跃写入者的限制，再查内存调度器、锁、WebSocketHub 与团队任务状态的所有权。数据库能接受连接，不等于业务组件已完成分布式协调。